Сравнение специализированных локальных LLM, general-моделей и SAST — по отдельности и в гибриде — для снижения ложных срабатываний и создания постоянных артефактов безопасности во встроенном C/C++ (IoT)

Проблема

В встроенном C/C++ коде найденную дыру нечем закрепить. Анализатор не видит контекста прошивки: условная компиляция через `#ifdef` и Kconfig, свои протоколы, специфичные для железа баги. Из-за этого он тонет в ложных срабатываниях, и такой инструмент в итоге просто отключают. Модель тоже не спасает: даже та, что обучена на уязвимостях и работает локально, каждый прогон отвечает по-другому. Она печатает отчёт в консоль, а после неё в репозитории не остаётся ничего. В облако не уйдёшь — код прошивки не должен покидать доверенный контур.

Что говорит литература

Связка анализатора и модели работает лучше одного анализатора

1

IRIS с GPT-4: из 120 известных уязвимостей Java находит 55, один CodeQL — 27, и ложных среди поднятого на 5 процентных пунктов меньше[9]. На 63 уязвимостях 10 проектов C# общая оценка полноты и точности 0.75–0.80 у моделей против 0.26–0.55 у анализаторов — но шуму больше[11]. Вывод живёт только до конца анализа.

Источники: **[9]** IRIS (Li, Dutta, Naik) · **[11]** Gnieciak & Szandala, IEEE Access 2025

Модель умеет писать исполняемые вещи, а не только текст

2

Доказательство уязвимости исполнением: CVE-Genie воспроизводит 428 из 841 CVE[14]; у PoC-Gym из 338 прогонов запуск проходят 116, но в нужное место попадают лишь 65 — это 12 из 20 CVE[15]. Запросы к анализатору: годные для 53.4% из 176 CVE против 10% без проверки запуском[7].

Источники: **[14]** CVE-Genie (Ullah et al.) · **[15]** PoC-Gym (Gezgin et al.) · **[7]** QLCoder

Специализированные модели по уязвимостям работают локально

3

VulnLLM-R-7B в агентской обёртке обходит CodeQL и переборщик AFL++ и находит ещё не закрытые дыры[1]. R2Vul-1.5B умеет рассуждать: найти → объяснить → указать место[2]. CodeAstra — 0.5 млрд параметров и вердикт «уязвимо / CWE»[3]. Все три на одном ноутбуке.

Источники: **[1]** VulnLLM-R (Nie et al.) · **[2]** R2Vul · **[3]** CodeAstra

Пробел: находку не закрепляют в проекте

4

Проверку, которая живёт в коде после выключения модели, для встроенного C/C++ пока не сделал никто. LLIoT показал, что модель находит CVE в прошивках, но это разовый эксперимент[16]; массовый перенос находки в правило есть только для залатанного кода и облачных моделей[41].

Источники: **[16]** LLIoT (Rezaei et al.) · **[41]** Autogrep

Как эта проблема решается в других работах

РАБОТА	ЧТО ДЕЛАЮТ	НЕДОСТАТКИ И ОГРАНИЧЕНИЯ
Sift[8], **IRIS**[9], **AdaTaint**[10]	агент разбирает предупреждения CodeQL: ложных стало меньше с >92% до 6.3%[8]	предупреждение закрыли — в репозитории ничего не осталось
QLCoder[7]	из CVE собирают запрос к анализатору: годный для 53.4% из 176 CVE, без проверки запуском — 10%[7]	Java, облачная модель, один вид проверки
Autogrep[41]	645 правил Semgrep из 39 931 патчей; отбор тот же: сработал на уязвимом, не сработал на исправленном[41]	нужен готовый патч; модель в облаке; только правила
Cleverest[13]	тесты по изменению коммита: 72 коммита в 8 проектах, 2 минуты против 24 часов фаззинга[13]	цель «не сломать прежнее», а не «дыра не вернулась»
CVE-Genie[14], **PoC-Gym**[15]	воспроизводят доказательство уязвимости исполнением: 428 из 841 CVE (~51%), у PoC-Gym — 12 из 20 Java-CVE[15]	разовое доказательство, не постоянная проверка в CI
VulnLLM-R[1], **CodeAstra**[3], **LLIoT**[16]	локальные модели находят CVE в C; разбор 15 116 CVE по IoT: чаще всего ломается память[16]	вывод живёт до конца прогона — закреплять не пытались

Чем я отличаюсь

У меня на вход — находка **локальной модели**, а не чужой залатанный коммит: у Autogrep и QLCoder патч уже есть. Не одно правило, а **пять видов проверок**. Встроенный C/C++ — это ещё и сборка, поэтому я смотрю, **собирается ли код** при таких флагах, и тестирую без UART и I2C. **Срок жизни проверки**: ловит ли она более новые баги.

Research Questions

RQ	ВОПРОС	ГЛАВНАЯ МЕТРИКА ОТВЕТА
RQ1	Кто лучше находит настоящие CVE в встроенном C/C++ коде: специализированная модель, обычная большая модель, анализатор или их гибрид?	полнота и точность поиска, плюс сколько ложных при равной полноте
RQ2	Какая комбинация локальных моделей (специализированные детекторы + general coder + explainer) производит **валидные артефакты** (правила, тесты, harness), которые работают **без LLM** на будущих коммитах?	**AVR — доля хороших артефактов: сколько из сгенерированного реально работает**
RQ3	Переносится ли умение на более новые CVE — или модель заучивает ответ?	не падает ли AVR на свежих CVE

Подход к решению

Шаг 0–1	Сбор корпуса CVE по 8 проектам для встроенных устройств, без повторов одного класса	≈170 CVE
Шаг 2 · фильтр	Semgrep по опасным функциям — он детерминирован, своего набора не теряет	дешёвая проверка спереди, дорогая модель — только на кандидатах
Шаг 3 · отбор	Маленькая модель CodeAstra [3] ×5 прогонов, ответы складываются	цель: ≥95% находок
Шаг 4 · анализ	VulnLLM-R-7B [1,12] ×30 прогонов, повторы совмещают по файлу, функции и классу CWE	покрытие ≈97%
Шаг 5 · закрепление	Находка превращается в проверки: шаблон, тест, перебор входов, заглушка железа, отчёт об уязвимости	[7,13] каждый тип пишет своя модель
Шаг 6 · отбор хороших	Проверка обязана сработать на уязвимой версии и не сработать на исправленной	[7,14] здесь считается AVR
Шаг 7	Хорошие проверки уезжают в git и работают в CI на каждом коммите — уже без модели	

Как построено исследование

Схема такая: проблема, цели, проект конвейера, сборка на реальном коде, оценка. Модели не дообучаю.

С чем сравниваю

Четыре условия на одном корпусе: A — только специализированная модель, B — только обычная, C — только анализатор, D — гибрид.

Почему 30 прогонов

Ответ модели неустойчив, поэтому опыт повторяю 30 раз; условия сравниваю парными тестами на тех же CVE.

Артефакты: что остаётся после модели

АРТЕФАКТ	ЧТО ЭТО	КАК ПРОВЕРЯЕТСЯ БЕЗ МОДЕЛИ
Semgrep rule security/rules/*.yaml	Паттерн опасного места: `memcpy`, `strcpy`, `alloca` без проверки длины. Ловит класс багов	Срабатывает на уязвимом, не срабатывает на исправленном, 0 ложных на 100 чистых файлах
CodeQL query[7,9] security/queries/*.ql	Запрос по дереву программы и потоку данных: грязный вход → опасная функция, видит путь между функциями	Тот же критерий на паре коммитов
Regression test[13] security/tests/*.c + ASan	Тест с подобранным adversarial-входом; ловит конкретный экземпляр — например переполнение счётчика длины в wolfCrypt CMAC	`gcc -fsanitize=address`: падает на уязвимом коммите, проходит на залатанном
Fuzzing harness[14,15] security/harness/*.cpp	Точка входа для переборщика случайных данных: разбор URI CoAP, TLS-рукопожатие	`clang -fsanitize=fuzzer`: воспроизводит краш за 3–5 мин
Hardware mock security/mocks/*.c	Заглушки I2C / SPI / UART / GPIO с той же сигнатурой — тест без платы. Mock не «чинит» баг, а эмулирует сырое поведение устройства	Сборка с `-DMOCK_HARDWARE`: тест падает там, где должен, и одинаково при повторе
Advisory · compliance map[5] security/advisories/*.md	Описание дыры в GHSA-формате с CVSS-вектором + привязка к CWE / MISRA / DO-178C	Ручная экспертиза двух человек

Встраивание в CI/CD: двойной режим

Быстрый контур — каждый коммит

Что работает	шаблоны Semgrep по изменённым строкам и по всему дереву; тесты с проверкой памяти, собранные вместе с проектом; обёртки для перебора входов — по 5 секунд на точку входа
Время	≤ 60 с — столько же, сколько сама сборка
Стоимость	≈ 0 ₽ на коммит: затраты модели оплачены один раз, при закреплении
Результат	упал тест — сборка красная, коммит не проходит
Особенность	ни моделей, ни сети: проверки — обычные файлы в `git`, результат одинаков от запуска к запуску

Ночной контур — раз в сутки или в неделю

Что работает	дешёвые шаблоны → отбор маленькой моделью ×5 → глубокий анализ ×30 → создание проверок → отбор запуском[8]
Время	часы, но вне цикла коммита — разработчик модель не ждёт
Стоимость	секунды видеокарты и рубли, один раз на находку
Результат	новая подтверждённая находка → новая проверка в папку `security/` через pull request
Ещё запуски	вне очереди, когда в базе CVE появляется свежая дыра в зависимости проекта; база растёт только хорошими проверками, модели не переобучаются

Датасеты

ПРОЕКТ (C/C++)	CVE ≈	СВЕЖИЕ CVE	ПОЧЕМУ
wolfSSL CVE-2026-5477 — переполнение в CMAC	40–45	есть	TLS-стек для IoT, память и разбор пакетов
mbed-TLS	20–25	есть	эталон условной компиляции
RIOT-OS · Zephyr CVE-2020-10063 — CoAP	22 · 30	2024–25	ОС для IoT; сетевые стеки и драйверы
Contiki-NG · OpenThread · NuttX · libcoap	5–20	по-разному	RPL / Thread / RTOS / CoAP
Итого основной корпус	**≈170**	старые / свежие	каждый CVE = пара коммитов: уязвимый и исправленный[22]

Свежие CVE: зачем проверять на них

На старых уязвимостях модель может не находить дыру, а **вспоминать** готовый ответ[28]. Поэтому старые CVE идут только на настройку промптов и порогов, а **все метрики считаю на более свежих** — тех, которых модель видеть не могла. Не падает там AVR — навык перенёсся, падает — модель помнит ответ. Это ответ на RQ3.

Не идёт в выборку: Juliet[27], SastBench[21], SecVulEval[20] — синтетика · PrimeVul[22], Devign[26] — утечка обучения · **резерв:** SVEN, ARVO, CleanVul[23].

Модели — все локальные, код не уходит в облако

МОДЕЛЬ	ПАМЯТЬ	ПРОГОНОВ	РОЛЬ В КОНВЕЙЕРЕ	НА ЧЁМ УЧИЛАСЬ · ОЦЕНКА
CodeAstra-500M[3] fp16	1 ГБ	×5	Быстрый фильтр после Semgrep — дешёвый первый эшелон	140 335 строк кода, в основном C (Linux, Chromium, PHP); вердикт: уязвимо / CWE
VulnLLM-R-7B[1] Q4	5 ГБ	×30	Глубокий анализ; тесты с проверкой памяти	PrimeVul + ARVO + Juliet + SVEN; в обёртке-агенте обходит CodeQL и AFL++
Qwen2.5-Coder-7B Q4	5 ГБ	×3	Пишет код проверок: шаблоны, обёртки для перебора, заглушки железа	обычная модель-кодер — контрольная группа: та же задача без специализации
Foundation-Sec-8B[5] Q4	6 ГБ	×3	Документация: отчёты об уязвимости, карты соответствия (MITRE, CWE, ГОСТ)	Llama-3.1-8B + 5.1 млрд токенов кибербеза
R2Vul-1.5B[2] · **Llama-3.1-8B**	1–5 ГБ	база	Второе мнение (ответы складываю) и базовые сравнения	обучение на готовых рассуждениях; настройка порога вердикта — как у моего фильтра

- **Одна модель — одна задача**: проверку пишет своя модель, а отбирает хорошие — детерминированная проверка. Без неё у QLCoder[7] 10% вместо 53.4%, а в Cleverest[13] тест доводит сборка и запуск.
- **Ансамбль, а не один прогон**: ни одному вердикту не доверяю — Semgrep детерминирован, фильтр прогоняю ×5 и складываю ответы, аудит ×30 с совмещением повторов.

Метрики поиска уязвимостей (ответ на RQ1)

Сравниваю только при равных условиях: один и тот же корпус, одинаковые пороги и одинаковое число прогонов.

Полнота поиска

Сколько из 170 известных CVE модель назвала: TP / (TP + FN).

TP — пойманная дыра, FN — пропуск. Попадание = та же функция, что залатали.

Точность

Сколько из названного — настоящие дыры: TP / (TP + FP).

FP — ложное срабатывание: указала на функцию, которая в этом коммите не уязвима.

Сколько шума
ПРИ РАВНОЙ ПОЛНОТЕ

Сколько отчётов читать ради одного настоящего бага.

Сравнение только при равной полноте: иначе «меньше шума» получается пропуском всего.

Ценой скольких прогонов

Не порезал ли дешёвый фильтр находки и сколько повторов нужно.

Где кривая уникальных находок выравнивается — там и бюджет повторов.

100%
детерминированный уровень (Semgrep)

≥ 95%
фильтр не режет находки

≈ 97%
полнота аудита за 30 прогонов

≥ 85%
сквозная полнота конвейера

Метрики закрепления: AVR и перенос навыка (ответ на RQ2–RQ3)

AVR — главная метрика работы (RQ2)

```
AVR = хорошие проверки / все сгенерированные
```

Хорошая = шаблон собрался, сработал на уязвимой версии, не сработал на исправленной и не дал ложных на 100 чистых файлах. Для ориентира в литературе: 53.4% у QLCoder[7] и ≈51% у CVE-Genie[14]. На локальных моделях правдоподобно ждать 40–60%, а 90% — верный признак, что модель помнит ответ.

Покрытие класса

RQ2

Поймает ли проверка другие CVE того же типа, хотя выводилась по одному.

Понятно, ловит ли она общий признак уязвимости, а не запомненный кусок кода[9].

Свежие CVE

RQ3

Не падает ли AVR на более новых багах.

AVR на свежих против AVR на старых: ≈1 = навык перенёсся, <0.5 = модель помнит ответ из весов[28,21,23].

Литература · специализированные модели и гибриды LLM+SAST

1. **Y. Nie et al.** VulnLLM-R: Specialized Reasoning LLM with Agent Scaffold. arXiv:2512.07533, 2025.
   arxiv.org/abs/2512.07533
2. **M. Weyssow et al.** R2Vul: Learning to Reason about Vulnerabilities with RL and Distillation. arXiv:2504.04699, 2025.
   arxiv.org/abs/2504.04699
3. **H. S. Ganesan** CodeAstra-500M / 7B: Laptop-Scale Vulnerability Detection. HF model cards, 2026.
   huggingface.co/rootxhacker/codeastra-500M
4. **S. Vijay et al.** Antares: Foundation Models for Agentic Vulnerability Localization (IBM). arXiv:2608.02407, 2026.
   arxiv.org/abs/2608.02407
5. **Cisco Foundation AI** Foundation-Sec-8B: An Open-Source LLM for Cybersecurity. 2025.
   huggingface.co/fdtn-ai/Foundation-Sec-8B
6. **AI Security Assistant** CyberAssist-mini: Lightweight LLM for Vulnerability Knowledge. GitHub, 2025.
   github.com/Ai-Security-assistant/CyberAssist
7. **X. Wang et al.** QLCoder: A Query Synthesizer for Static Analysis of Security Vulnerabilities. arXiv:2511.08462, 2025.
   arxiv.org/abs/2511.08462
8. **Y. Xiong, T. Zhang** Sift: A Comparative Study of LLM Agents in Vulnerability False Positive Filtering. ISSTA 2026.
   arxiv.org/abs/2601.22952
9. **Z. Li, S. Dutta, M. Naik** IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. arXiv:2405.17238, 2024.
   arxiv.org/abs/2405.17238
10. **S. Lin** AdaTaint: LLM-Driven Adaptive Source–Sink Identification and False Positive Mitigation. arXiv:2511.04023, 2025.
    arxiv.org/abs/2511.04023

Литература · генерация тестов/PoC и IoT

11. **D. Gnieciak, T. Szandala** Large Language Models Versus Static Code Analysis Tools: Systematic Benchmark. IEEE Access 13:198410–198422, 2025.
doi.org/10.1109/ACCESS.2025.3635168

12. **K. Tamberg, H. Bahsi** Harnessing LLMs for Vulnerability Detection: CoT / ToT / Self-Consistency. IEEE Access 13:29698–29717, 2025.
doi.org/10.1109/ACCESS.2025.3541146

13. **J. Liu, S. Lee, E. Losiouk, M. Böhme** Evaluating LLM-Based Regression Test Generation (Cleverest). Proc. ACM Softw. Eng., 2026.
doi.org/10.1145/3808129

14. **S. Ullah et al.** From CVE Entries to Verifiable Exploits: Multi-Agent Framework for Reproducing CVEs (CVE-Genie). arXiv:2509.01835, 2025.
arxiv.org/abs/2509.01835

15. **D. Gezgin et al.** PoC-Gym: Towards More Reliable LLM-Assisted Proof-of-Concept Exploit Generation. arXiv:2602.04165, 2026.
arxiv.org/abs/2602.04165

16. **M. Rezaei et al.** SoK: Understanding the State of IoT-Specific Vulnerabilities (LLIoT). 2026.
conand.me/publications/rezaei-lliot-2026.pdf

17. **I. Khasanov, E. Sadovykh** Improving Vulnerability Analysis of Docker Images. ACM Ada Letters.
doi.org/10.1145/3821489.3821505
+ датасет
zenodo.15301809

18. **I. Khasanov, E. Sadovykh** Evaluating DAVS Approach for Docker Images Static Analysis. ICCSA, Springer.
doi.org/10.1007/978-3-032-00644-8_11

19. **T.-P. Doan, S. Jung** DAVS: Dockerfile Analysis for Container Vulnerability Scanning. CMC 72(1), 2022.
techscience.com/cmc/v72n1/46926

20. **M. B. U. Ahmed et al.** SecVulEval: Benchmarking LLMs for Real-World C/C++ Vulnerability Detection.
arxiv.org/abs/2505.19828

Литература · бенчмарки и датасеты

21. **J. Feiglin, G. Dar** SastBench: A Benchmark for Testing Agentic SAST Triage.
arxiv.org/abs/2601.02941

22. **Y. Ding et al.** PrimeVul: Vulnerability Detection with Code LMs — How Far Are We? ICSE 2025.
arxiv.org/abs/2403.18624

23. **Y. Li et al.** CleanVul: Function-Level Vulnerability Detection in Code Commits Using LLM Heuristics.
arxiv.org/abs/2411.17274

24. **J. Fan et al.** Big-Vul: A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. MSR 2020.
huggingface.co/datasets/bstee615/bigvul

25. **Y. Chen et al.** DiverseVul: A New Vulnerable Source Code Dataset. RAID 2023.
arxiv.org/abs/2304.00409

26. **Y. Zhou et al.** Devign: Effective Vulnerability Identification via GNNs. NeurIPS 2019.
arxiv.org/abs/1909.03496

27. **NIST** SAMATE Juliet Test Suite.
samate.nist.gov/SARD

28. **S. Chakraborty et al.** Deep Learning Based Vulnerability Detection: Are We There Yet? IEEE TSE, 2021.
arxiv.org/abs/2009.07235

29. **H. Cui et al.** Empirical Study of False Negatives/Positives of Static Code Analyzers.
arxiv.org/abs/2408.13855

30. **M. Christakis, C. Bird** What Developers Want and Need from Program Analysis. ASE 2016.
doi.org/10.1145/2970276.2970347

Литература · обзоры, CI/CD и генерация правил

31. **Z. Sheng et al.** LLMs in Software Security: A Survey of Vulnerability Detection Techniques.
arxiv.org/abs/2502.07049

32. **S. Kaniewski et al.** SLR on Detecting Software Vulnerabilities with LLMs (263 studies).
arxiv.org/abs/2507.22659

33. **M. G. Tehrani et al.** LLM vs. SAST: GPT-4 vs SonarQube, 32 сценария, McNemar.
arxiv.org/abs/2506.15212

34. **B. Koishybayev et al.** Угрозы CI/CD в GitHub Actions: анализ 447K workflow. USENIX Security 2022.
usenix.org/sec22-koishybayev

35. **Argus**: taint-анализ CI/CD workflow. USENIX Security 2023.
usenix.org/usenixsecurity23-muralee

36. **G. Benedetti, L. Verderame, A. Merlo** Automatic Security Assessment of GitHub Actions Workflows (GHAST), 2022.
arxiv.org/abs/2208.03837

37. **M. Fares, Y. Gamage, B. Baudry** Unpacking Security Scanners for GitHub Actions Workflows (9 сканеров).
arxiv.org/abs/2601.14455

38. **D. Dubniczky et al.** CASTLE: 250 C-программ, 13 анализаторов × 10 LLM.
arxiv.org/abs/2503.09433

39. **K. Li, S. Chen et al.** SAST Tools for Java: Comparison and Evaluation. FSE 2023.
sen-chen.github.io

40. **Y. Zhang et al.** How Can ChatGPT Support Human Security Testers to Mitigate Supply Chain Attacks? (24 доказательства, 4 CVE).
arxiv.org/abs/2310.00710

41. **Autogrep** (Lambda Security): Automated Generation and Filtering of Semgrep Rules from Vulnerability Patches, 2025.
github.com/lambdasec/autogrep

Спасибо за внимание